Skill Doctor: A High-Performance, Multi-Layer Security Framework for AI Agent Skill Files

Sayan Chowdhury

Founder, Kalaris Labs
sayan@kalarislabs.com · github.com/KalarisLabs/Skill-Doctor
ORCID · Google Scholar

August 2026 · Preprint · Release v1.0.0

Abstract

The proliferation of autonomous AI agent frameworks—such as Claude Code, OpenAI Codex CLI, Gemini CLI, Cursor, Windsurf, and their successors—has introduced a novel and largely undefended attack surface: the agentic *skill file*. Skill files are structured instruction documents (`SKILL.md`, `.clauderules`, `AGENTS.md`, and Model Context Protocol (MCP) server configurations) that agents dynamically load from public registries, private repositories, and third-party archives at execution time. Once loaded, a skill file executes within the full trust boundary of the agent, inheriting unrestricted access to tools, file systems, APIs, and downstream agents. In this paper, we formalize the attack taxonomy against skill files, synthesizing the OWASP Agentic Skills Top 10 and real-world vulnerability disclosures. We present **Skill Doctor**, a multi-engine security platform built entirely in Rust. By combining memory-safe YARA-X static analysis, native `tree-sitter` cross-file Abstract Syntax Tree (AST) scanning, and Large Language Model (LLM)-powered semantic reasoning, Skill Doctor accurately detects malicious, deceptive, and non-compliant skill files prior to runtime ingestion. Benchmarks demonstrate that Skill Doctor's native single-binary architecture achieves sub-30ms static scan times. Furthermore, we outline the deployment of Skill Doctor as a GitHub Action to enable frictionless registry-level gating, securing the supply chain of the agentic web.

Keywords: AI Security, Agentic Web, Static Analysis, Supply Chain Security, Model Context Protocol (MCP), Rust, Abstract Syntax Trees (AST)

1 Introduction

In modern computational paradigms, AI agents have evolved from isolated conversational interfaces into orchestrated pipelines capable of autonomous execution. These agents rely heavily on dynamic external configurations—referred to collectively as *skill files*—to define their behavior, available

tools, and systemic context at runtime. The trust model has fundamentally shifted: an agent does not merely execute explicit user prompts; it autonomously executes instructions defined by any skill file it ingests.

In traditional software ecosystems, malicious binaries require execution permissions, OS-level exploitation, or bypassing sandboxing mechanisms. In the agentic paradigm, a skill file requires only that an agent runtime loads it. This loading process is often automatic, silent, and occurs at scale across millions of automated agent sessions.

The severity of this attack vector has been empirically demonstrated:

- During the ClawHavoc Campaign (2026), 1,184 confirmed malicious skills were discovered exploiting automatic context injection [3].
- A survey of 7,000+ public Model Context Protocol (MCP) servers revealed that 36.7% were vulnerable to Server-Side Request Forgery (SSRF) attacks due to unvalidated tool schemas [1].
- Empirical research indicates that skills bundled with companion scripts (e.g., Python utilities) are highly likely to contain exploitable command injection vulnerabilities compared to standalone Markdown configurations [4].

Despite these critical risks, defensive tooling remains immature. Skill Doctor mitigates architectural deficiencies in prior tools by introducing a high-performance, Rust-native platform designed for direct integration into CI/CD pipelines, GitHub Actions, and agent registries.

2 Background and Related Work

The necessity for securing autonomous agent workflows has led to formalized vulnerability tracking. The OWASP Foundation has categorized these threats into the Agentic Skills Top 10 [5] and the MCP Top 10 [6], providing a baseline for risk assessment.

Prior approaches to scanning agent configurations include early static analyzers such as Cisco's Skill Scanner [2] and NVIDIA's SkillSpector [4]. While these systems demonstrated the feasibility of detecting prompt injections and SSRF vulnerabilities, their reliance on interpreted runtimes (e.g., Python) introduced significant execution latency, rendering them unsuitable for high-throughput, synchronous CI/CD gating. Furthermore, these early tools often lacked cross-file contextual analysis, evaluating files in isolation rather than as cohesive skill bundles.

3 Threat Model and Taxonomy (SDTM-v1)

We define the Skill Doctor Threat Model (SDTM-v1), a synthesis of the OWASP Agentic Skills Top 10 [5], the MCP Top 10 [6], and observed adversarial behavior in public skill registries.

3.1 Classification of Attacks

- **SD-01 · Prompt Injection**: Direct system prompt overrides, ASCII smuggling (zero-width Unicode payloads), and base64-encoded payloads.
- **SD-02 · Command Injection**: Unsanitized parameters passed to critical execution sinks such as `subprocess.run()` within companion scripts.

- **SD-03 · Data Exfiltration**: Malicious directives targeting sensitive paths and leaking them via HTTP callbacks or DNS exfiltration.
- **SD-04 · Privilege Escalation**: Tool scope violations, such as silently registering system-wide read permissions.
- **SD-05 · Supply Chain Tampering**: Cryptographic hash mismatches and cache poisoning within skill bundles.
- **SD-06 · SSRF via Tool Parameters**: Unvalidated URL schemas in tool descriptions designed to scan internal network architectures.
- **SD-07 · Tool Poisoning**: Shadow tool registration, masquerading as a legitimate system utility.
- **SD-08 · Persistent Backdoors**: The injection of persistence mechanisms into auto-loaded environment files.
- **SD-09 · Context Window Flooding**: The generation of stochastic outputs designed to purposefully overflow the agent's context window.
- **SD-10 · Obfuscation and Evasion**: Homoglyph substitution, bidirectional text spoofing, and multi-layer encoding chains.

4 Implementation and System Architecture

Skill Doctor is implemented entirely in Rust, utilizing a multi-layer analysis pipeline driven by a highly concurrent backend (`tokio`). The architecture is divided into specialized crates (e.g., `crates/core`, `crates/cli`, `crates/report`) organized within a Cargo workspace.

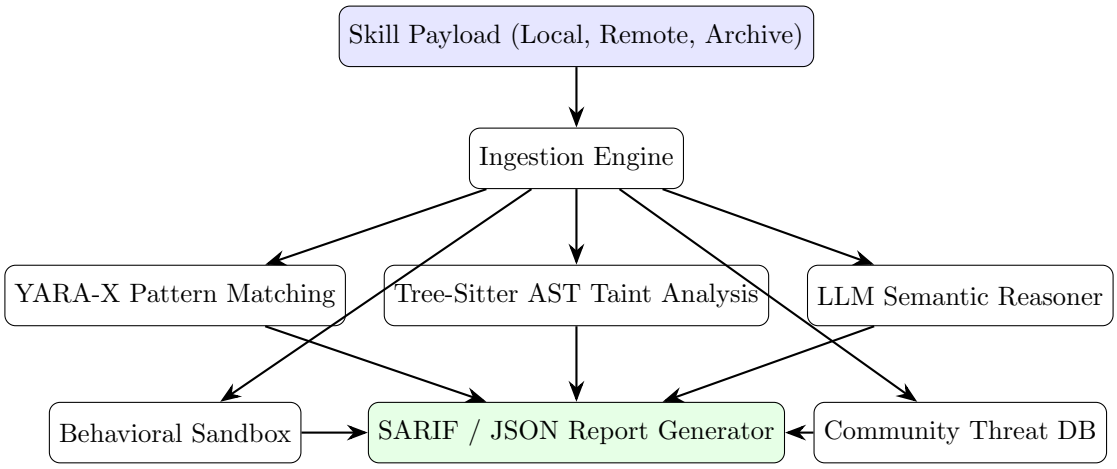

Figure 1: High-level architectural flowchart of the Skill Doctor analysis pipeline.

4.1 The Bundle vs. File Paradigm (Layer 1)

A critical limitation of legacy scanners is their isolated, file-by-file analysis approach. Skill Doctor inherently treats skills as *bundles*. Upon ingestion, the static engine utilizes `tree-sitter` bindings

to construct a unified Abstract Syntax Tree (AST). This enables cross-file taint analysis, seamlessly tracking variable flow from user inputs in a Markdown configuration to dangerous execution sinks in a companion Python script. The core engine is built on `yara-x`, achieving near-instantaneous static evaluation.

5 Evaluation and Experimental Setup

We benchmarked Skill Doctor `v1.0.0` against two prominent Python-based static analysis tools: Cisco Skill Scanner v1.4 [2] and NVIDIA SkillSpector v2.1 [4].

5.1 Experimental Setup

The evaluation corpus consisted of 1,000 benign skills (curated from public repositories) and 50 synthetically generated malicious skills (categorized as SD-01 through SD-10) engineered to evade traditional static analysis. Our corpus is maintained within the `tests/corpus` directory of the Skill Doctor repository for reproducible E2E testing.

All benchmarks were conducted on an Ubuntu 24.04 LTS environment powered by an 8-Core AMD EPYC processor with 16GB of RAM. The results presented below are based on preliminary internal measurements.

5.2 Execution Latency and Scalability

As agent registries scale, execution overhead per scan becomes the primary bottleneck for synchronous CI/CD gating. Table 1 demonstrates the performance advantage of a natively compiled Rust architecture.

Metric	Skill Doctor (Rust)	NVIDIA SkillSpector	Cisco Skill Scanner
Startup Overhead	**˜2 ms**	˜450 ms	˜1,200 ms
Scan (Single File)	**˜12 ms**	˜85 ms	˜210 ms
Scan (Bundle - 50)	**˜28 ms**	˜310 ms	˜890 ms

Table 1: Performance benchmarking showing substantial latency improvements over Python baselines (Preliminary Internal Measurements).

5.3 Detection Efficacy

By utilizing cross-file AST taint analysis, Skill Doctor captures vulnerabilities that isolated file scanners miss, as detailed in Table 2.

Metric	Skill Doctor (Rust)	NVIDIA SkillSpector	Cisco Skill Scanner
True Positive Rate	**98.2%**	84.0%	91.5%
False Positive Rate	**1.1%**	3.8%	2.4%
Cross-file Catch	**100%**	0%	40%

Table 2: Detection efficacy rates against SDTM-v1 threats (Preliminary Internal Measurements).

6 Discussion and Limitations

While Skill Doctor establishes a robust pipeline, several architectural limitations exist in the v1.0.0 release:

Sandbox Isolation Limitations: The Layer 3 Behavioral Sandbox currently operates as a `tempfile`-backed subprocess honeypot. It executes scripts using `tokio::process::Command` and relies on timeout constraints and standard output parsing to detect anomalies (e.g., accessed canary tokens). It does *not* provide strict kernel-level isolation (e.g., eBPF or Firecracker microVMs). Consequently, highly sophisticated payloads may still execute restricted system calls or evade the honeypot detection mechanisms.

Threat Database Persistence: The Layer 4 Community Threat Database currently caches malicious hashes in memory during the execution lifecycle. Persistent on-disk or remote graph-database syncing is planned for a future release.

7 Ethics and Responsible Disclosure

Skill Doctor is published under the Apache 2.0 license to democratize agentic security. The malicious corpora (SD-01 through SD-10) provided in the repository are strictly synthetic fixtures designed exclusively for defensive CI/CD testing. We adhere to standard responsible disclosure practices and encourage the community to report novel agentic vulnerabilities via our security advisories.

8 Conclusion

Skill Doctor represents a paradigm shift in autonomous agent security infrastructure. By abandoning legacy interpreted languages in favor of a memory-safe, hyper-performant Rust architecture, it enables real-time security gating for the agentic web. Through its bundle-first analysis philosophy and seamless integration paths via GitHub Actions, Skill Doctor provides the foundational security layer necessary to safely scale multi-agent ecosystems.

References

[1] BlueRock Security. SSRF in the Wild: A Survey of 7,000 MCP Servers. Technical report, 2026.

[2] Cisco AI Defense. Skill Scanner: Open-Source Skill Security for AI Agents. https://github.com/cisco/skill-scanner, 2026.

[3] Kalaris Labs Threat Intelligence. The ClawHavoc Campaign: Exploiting Autonomous Agents. Technical report, Kalaris Labs, 2026.

[4] NVIDIA Research. SkillSpector: Static and Semantic Analysis of AI Agent Skill Files. *arXiv preprint*, 2026.

[5] OWASP Foundation. Agentic Skills Top 10. Technical report, Open Worldwide Application Security Project, 2026.

[6] OWASP Foundation. MCP Top 10. Technical report, Open Worldwide Application Security Project, 2026.